

廣東工業大學

《计算机网络》课程设计报告

学　　院　国际教育学院

专　　业　计算机科学与技术

年级班别　24 级计科 1 班

学　　号　3124009837

学生姓名　黄建刚

指导教师　梁路

成　　绩

网络嗅探器课程设计报告

一、课程设计题目

基于原始套接字的网络嗅探器

二、实验环境及工具

1. 开发环境：Visual Studio Code（Trae IDE）
2. 操作系统：Windows 10/11
3. 程序设计语言：Python 3.6+
4. 依赖库：仅使用Python标准库（无需安装任何第三方库）
   - `socket` — 原始套接字创建与网络通信
   - `struct` — 二进制数据包解析
   - `tkinter` — 图形用户界面构建
   - `threading` — 多线程数据包捕获
   - `ctypes` — Windows API调用（管理员权限检测）
   - `datetime` — 时间戳记录
   - `time` — 时间计算
   - `subprocess` — 调用系统Ping命令

三、已知技术参数和设计要求

1. 实现一个图形用户界面的局域网网络数据包嗅探工具。
2. 功能：利用原始套接字捕获流经本机网卡的所有数据包，并逐层解析。
   2.1 数据包捕获：将网卡设置为混杂模式，接收所有流经网卡的数据包，不限于发给本机的数据包。
   2.2 IP层分析：解析IP头部，获取版本号、头部长度、TTL、源IP地址、目的IP地址、协议类型。
   2.3 传输层分析：根据IP头部的协议字段，分别对TCP（协议号6）、UDP（协议号17）、ICMP（协议号1）进行解析，获取端口号、序列号、标志位等信息。
   2.4 数据包详情展示：采用类似Wireshark的三栏布局，包含数据包列表、协议树详情、十六进制转储。
   2.5 协议筛选与搜索：支持按ALL/TCP/UDP/ICMP筛选数据包，以及关键词实时搜索。
   2.6 数据导出：将捕获的数据包导出为文本文件，便于后续分析。
   2.7 手动Ping工具：内置Ping功能，可向指定IP地址或域名发送ICMP Echo Request数据包，主动产生ICMP流量，确保嗅探器能够捕获到ICMP数据包。
3. 用户界面：包含控制按钮（开始/停止/清空/导出）、Ping工具面板、协议筛选、搜索框、统计面板、数据包列表、协议详情树、十六进制转储区。

四、原理

1、TCP（Transmission Control Protocol）

TCP（传输控制协议）是一种面向连接的、可靠的、基于字节流的传输层通信协议，由IETF的RFC 793定义。在简化的计算机网络OSI模型中，它完成第四层传输层所指定的功能。TCP层是位于IP层之上，应用层之下的中间层。不同主机的应用层之间经常需要可靠的、像管道一样的连接，TCP协议则提供这样的流机制。

1.1 TCP段格式

TCP头部固定长度为20字节，包含源端口、目的端口、序列号、确认号、数据偏移、控制位、窗口大小、校验和、紧急指针等字段。其中控制位包含6个标志：URG（紧急）、ACK（确认）、PSH（推送）、RST（重置）、SYN（同步）、FIN（结束）。

1.2 TCP三次握手

TCP建立连接需要经过三次握手：
- 第一次握手：客户端向服务器发送SYN包，请求建立连接
- 第二次握手：服务器收到SYN包后，回复SYN+ACK包，表示同意建立连接
- 第三次握手：客户端收到SYN+ACK后，发送ACK包确认，连接建立完成

1.3 TCP头部解析

在Python中使用struct模块解析TCP头部，通过格式字符串'!HHLLBBHHH'从20字节中解包出源端口、目的端口、序列号、确认号、数据偏移和标志位等信息。

```
def parse_tcp_header(self, data):
    if len(data) < 20:
        return None
    tcp_header = struct.unpack('!HHLLBBHHH', data[:20])
    source_port = tcp_header[0]          # 源端口
    dest_port = tcp_header[1]            # 目的端口
    sequence = tcp_header[2]             # 序列号
    acknowledgement = tcp_header[3]      # 确认号
    offset_res = tcp_header[4]           # 数据偏移+保留位
    tcp_offset = (offset_res >> 4) * 4 # 头部长度（单位4字节）
    flags = tcp_header[5]                # 控制位
    return source_port, dest_port, sequence, acknowledgement,
tcp_offset, flags, data[tcp_offset:]
```

1.4 TCP标志位解析

通过位运算解析TCP控制位，每个标志位对应一个二进制位：

```
def get_tcp_flags(self, flags):
    flag_str = []
    if flags & 0x01: flag_str.append('FIN')
    if flags & 0x02: flag_str.append('SYN')
    if flags & 0x04: flag_str.append('RST')
    if flags & 0x08: flag_str.append('PSH')
    if flags & 0x10: flag_str.append('ACK')
    if flags & 0x20: flag_str.append('URG')
    return ','.join(flag_str) if flag_str else 'None'
```

- 0x01: FIN（结束）
- 0x02: SYN（同步）
- 0x04: RST（重置）
- 0x08: PSH（推送）
- 0x10: ACK（确认）
- 0x20: URG（紧急）

2、UDP（User Datagram Protocol）

UDP（用户数据报协议）是OSI参考模型中一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。UDP在IP报文中的协议号是17。

2.1 UDP数据报格式

UDP头部只有8字节，包含源端口、目的端口、长度和校验和四个字段。UDP不保证数据报的可靠到达，但头部开销小、传输效率高，适用于视频通话、DNS查询等场景。

2.2 UDP头部解析

使用struct模块的'!HHHH'格式从8字节中解包出源端口、目的端口、长度和校验和。

```
def parse_udp_header(self, data):
    if len(data) < 8:
        return None
    udp_header = struct.unpack('!HHHH', data[:8])
    source_port = udp_header[0]    # 源端口
    dest_port = udp_header[1]      # 目的端口
    length = udp_header[2]         # UDP数据报长度
    checksum = udp_header[3]       # 校验和
    return source_port, dest_port, length, checksum, data[8:]
```

3、ICMP（Internet Control Message Protocol）

ICMP（互联网控制报文协议）是网络层协议，IP协议号为1，用于在IP网络中传递错误报告和网络诊断信息。ICMP报文封装在IP数据报的数据部分中。

3.1 ICMP报文格式

ICMP头部包含类型（8位）、代码（8位）和校验和（16位）三个字段。常见ICMP类型包括：

- 类型0：Echo Reply（Ping回复）
- 类型3：Destination Unreachable（目的不可达）

- 类型8：Echo Request（Ping请求）
- 类型11：Time Exceeded（TTL超时）

3.2 ICMP头部解析

直接从字节数组中按偏移位置提取ICMP类型和代码，使用struct.unpack解析校验和。

```
def parse_icmp_header(self, data):
    if len(data) < 8:
        return None
    icmp_type = data[0]           # 类型
    icmp_code = data[1]           # 代码
    checksum = struct.unpack('!H', data[2:4])[0]  # 校验和
    return icmp_type, icmp_code, checksum, data[4:]
```

4、多线程（Multithreading）

多线程是指从软件或硬件上实现多个线程并发执行的技术。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程，进而提升整体处理性能。

在本项目中，数据包捕获使用recvfrom()函数，该函数是阻塞操作——它会一直等待数据到来，期间无法执行其他代码。如果放在主线程中执行，会导致GUI界面卡死无法响应用户操作。因此采用多线程设计：

- 主线程（GUI线程）：负责界面渲染、按钮事件响应、统计面板更新
- 嗅探线程（工作线程）：负责原始套接字数据接收、协议解析、创建Packet对象

线程安全方面，由于Tkinter不是线程安全的，子线程不能直接操作GUI组件，因此使用root.after()方法将GUI更新操作调度到主线程执行。

5、原始套接字（Raw Socket）

原始套接字是网络编程中的一种特殊套接字类型，它允许应用程序直接访问网络层（IP层）的数据包，即直接接收和发送IP数据包，而不经过传输层的处理。这正是网络嗅探器能够捕获所有流经网卡数据包的技术基础。

5.1 原始套接字的创建

通过socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)创建原始套接字，其中AF_INET指定IPv4地址族，SOCK_RAW指定原始套接字类型，IPPROTO_IP表示接收所有IP数据包。

```
import socket
```

```
# 创建原始套接字
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
```

5.2 混杂模式

普通模式下，网卡只会接收目的MAC地址为本机的数据包。混杂模式（Promiscuous Mode）使网卡能够接收所有经过它的数据包，无论目的地址是什么。在Windows中通过Winsock控制码启用：

```
sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)       启用混杂模式
sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)     关闭混杂模式
sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)    # 启用混杂模式
# 捕获数据包...
sock.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)  # 关闭混杂模式
```

5.3 管理员权限

原始套接字可以捕获所有网络流量，包括其他进程的通信数据，涉及严重的隐私和安全问题。因此Windows系统要求必须以管理员身份运行程序才能创建原始套接字。本程序通过ctypes调用Windows API检测权限。

```
import ctypes

def is_admin():
    try:
        return ctypes.windll.shell32.IsUserAnAdmin()
    except:
        return False
```

6、struct模块与二进制数据解析

Python的struct模块用于在Python数据类型和C语言结构体之间进行二进制转换，是网络数据包解析的核心工具。网络协议中的头部都是二进制格式，使用struct可以方便地将字节流解析为可读的字段值。

6.1 格式字符串说明

- ！ — 网络字节序（大端序）
- B — unsigned char（1字节无符号整数）
- H — unsigned short（2字节无符号整数）
- L — unsigned long（4字节无符号整数）
- 4s — 4字节字符串

6.2 位运算解析

IP头部的第一个字节同时包含了版本号（高4位）和头部长度（低4位），需要通过位运算提取：

```
version_ihl = ip_header[0]
version = version_ihl >> 4              高4位：版本号
ihl = (version_ihl & 0xF)   4        低4位：头部长度（单位4字节）
version_ihl = ip_header[0]
version = version_ihl >> 4          # 高4位：版本号
ihl = (version_ihl & 0xF) * 4     # 低4位：头部长度（单位4字节）
```

五、设计与实现

1、设计思想

1.1 整体架构

本程序采用生产者-消费者模型设计：
- 生产者（嗅探线程）：通过原始套接字捕获网络数据包，逐层解析协议头部，生成Packet对象，存入数据包列表
- 消费者（GUI主线程）：从数据包列表中读取数据，更新Treeview列表、统计面板和详情面板

生产者和消费者通过共享列表self.packets进行数据传递，主线程通过after()方法安全地从列表中读取数据更新界面。这种设计使得数据包捕获和界面显示完全解耦，互不干扰。

1.2 分层解析策略

数据包解析遵循TCP/IP协议栈的分层模型，从原始字节流开始，从下到上逐层解封装：

原始字节流 -> IP层解析 -> 传输层解析
                                ├── TCP（协议号6）-> 端口、序列号、标志位
                                ├── UDP（协议号17）-> 端口、长度
                                └── ICMP（协议号1）-> 类型、代码

1.3 数据包存储设计

使用Packet类封装每个数据包的所有协议层信息，包含序号、时间戳、原始数据、以太网层信息、IP层信息、传输层信息和摘要信息，便于在列表和详情面板之间传递。

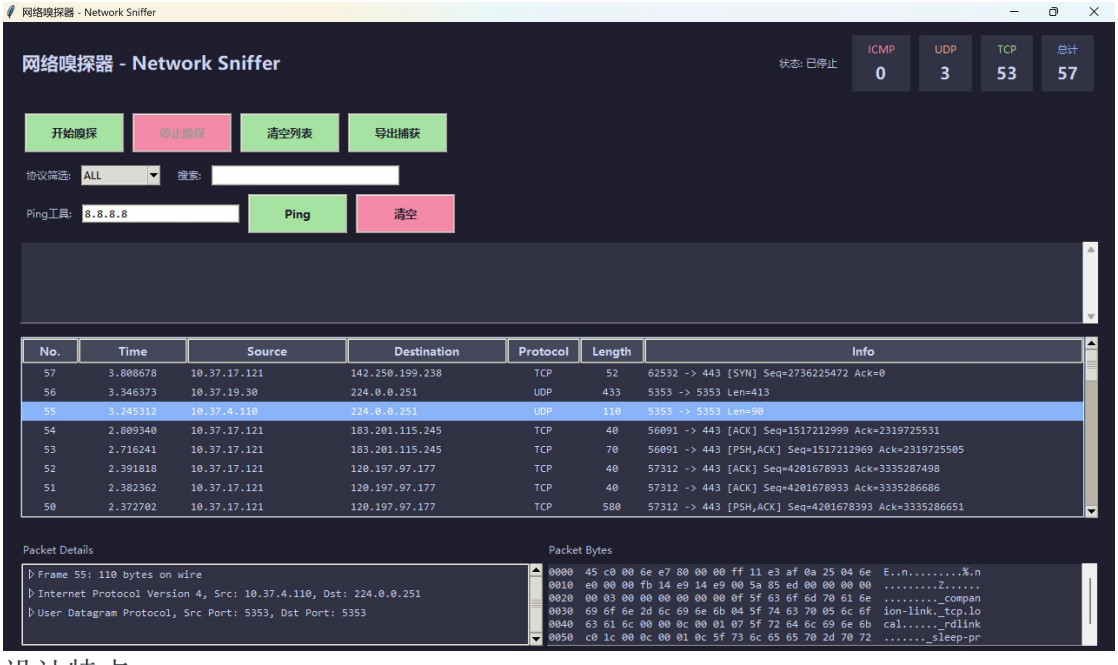
```
class Packet:
    def __init__(self, number, timestamp, relative_time,
raw_data,
                 eth_info, ip_info, transport_info, info):
        self.number = number           # 数据包序号
        self.timestamp = timestamp     # 捕获时间戳
        self.relative_time = relative_time  # 相对时间
        self.raw_data = raw_data       # 原始字节数据
        self.eth_info = eth_info       # 以太网层信息
        self.ip_info = ip_info         # IP层信息
        self.transport_info = transport_info  # 传输层信息
        self.info = info               # 摘要信息字符串
```

1.4 Ping工具设计

考虑到正常情况下ICMP数据包较少（通常只有主动Ping时才会产生），为了让嗅探器能够捕获到ICMP协议数据包，方便验证ICMP协议解析的正确性，程序内置了手动Ping工具。Ping工具调用subprocess模块执行系统的ping命令，由操作系统产生真正的ICMP Echo Request数据包，嗅探器的原始套接字可同时捕获Echo Request（类型8）和Echo Reply（类型0）数据包。Ping命令在独立线程中异步执行，不影响GUI操作。

2、界面设计

整体界面采用深色主题和类似Wireshark的三栏布局，从上到下分为四个区域：



设计特点：

- 深色主题：采用类似Wireshark的暗色配色（背景1e1e2e，卡片313244，文字cdd6f4）
- 三栏设计：数据包列表在上方，协议详情（左下）和十六进制转储（右下）在下方
- 实时统计：统计面板实时显示总计、TCP、UDP、ICMP数据包数量
- 协议筛选：支持下拉框筛选和关键词实时搜索

3、程序结构设计

3.1 程序目录结构

```
计算机网络课程设计/
│
├──── sniffer_gui.py                主程序（图形界面版本，包含所有功能）
├──── 启动网络嗅探器.bat             启动脚本（自动请求管理员权限）
└──── 课程设计报告.md                课程设计报告文档
```

3.2 类描述表格

类 / 函数	类型	说明
is_admin()	全局函数	检查是否以管理员身份运行
Packet	数据类	数据包数据模型，存储所有协议层信息
PacketSnifferGUI	主类	GUI 应用程序主类，包含所有界面和逻辑
init()	方法	初始化窗口、成员变量、调用界面构建
setup_ui()	方法	构建完整的图形用户界面
get_local_ip()	方法	获取本机局域网 IP 地址
parse_arp_header()	方法	解析 ARP 协议头部
parse_ip_header()	方法	解析 IP 协议头部
parse_tcp_header()	方法	解析 TCP 协议头部
parse_udp_header()	方法	解析 UDP 协议头部
parse_icmp_header()	方法	解析 ICMP 协议头部
get_protocol_name()	方法	IP 协议号转协议名称
get_tcp_flags()	方法	解析 TCP 控制位
generate_info()	方法	生成数据包摘要信息
start_sniffing()	方法	启动嗅探线程
stop_sniffing()	方法	停止嗅探
sniff_packets()	方法	核心嗅探循环，捕获和解析数据包
add_packet_to_tree()	方法	将数据包添加到 Treeview 列表
on_packet_select()	方法	数据包选中事件处理

类 / 函数	类型	说明
show_packet_detail()	方法	显示数据包详情
show_ip_detail()	方法	显示 IP 层详情
show_tcp_detail()	方法	显示 TCP 层详情
show_udp_detail()	方法	显示 UDP 层详情
show_icmp_detail()	方法	显示 ICMP 层详情
show_arp_detail()	方法	显示 ARP 层详情
show_hex_dump()	方法	显示十六进制转储
apply_filter()	方法	应用协议筛选和关键词搜索
update_stats()	方法	更新统计面板
start_ping()	方法	启动 Ping 工具
do_ping()	方法	执行 Ping 命令（子线程中）
clear_packets()	方法	清空所有数据包
export_capture()	方法	导出捕获数据到文本文件

4、关键程序说明

4.1 主程序入口

程序启动时，首先检测管理员权限。如果不是管理员身份，弹出错误提示并退出。通过ctypes.windll.shell32.IsUserAnAdmin()调用Windows API检测当前进程是否以管理员权限运行。

```
def main():
    if not is_admin():
        root = tk.Tk()
        root.withdraw()
        messagebox.showerror("权限错误",
            "请以管理员身份运行此程序！")
        sys.exit(1)

    root = tk.Tk()
    app = PacketSnifferGUI(root)
    root.mainloop()

if __name__ == '__main__':
    main()
```

4.2 数据包捕获流程

用户点击"开始嗅探"
-> start_sniffing() 创建嗅探线程
-> sniff_packets() 创建原始套接字

```
    -> 设置混杂模式（sock.ioctl）
    -> 循环调用 sock.recvfrom(65536) 接收数据包
    -> 解析协议头部（逐层解封装）
    -> 创建 Packet 对象
    -> 通过 after() 更新 GUI 界面
    -> 用户点击"停止嗅探"退出循环
    -> 关闭混杂模式，关闭套接字
def sniff_packets(self):
    host = self.get_local_ip()
    sock = None

    try:
        sock = socket.socket(socket.AF_INET,
                             socket.SOCK_RAW,
                             socket.IPPROTO_IP)
        sock.bind((host, 0))
        sock.setsockopt(socket.IPPROTO_IP,
                        socket.IP_HDRINCL, 1)
        sock.ioctl(socket.SIO_RCVALL,
                   socket.RCVALL_ON)

        while self.sniffing:
            try:
                raw_data, addr = sock.recvfrom(65536)
                self.packet_count += 1

                # 判断IP版本
                version = (raw_data[0] >> 4) & 0xF

                if version == 4:
                    # 解析IP头部
                    ip_info = self.parse_ip_header(raw_data)
                    if ip_info:
                        protocol = ip_info[3]
                        # 根据协议号分发解析
                        if protocol == 6:    # TCP
                            transport_info =
self.parse_tcp_header(ip_data)
                        elif protocol == 17: # UDP
                            transport_info =
self.parse_udp_header(ip_data)
                        elif protocol == 1:  # ICMP
                            transport_info =
self.parse_icmp_header(ip_data)

                # 创建Packet对象
                packet = Packet(self.packet_count, timestamp,
                                relative_time, raw_data,
                                eth_info, ip_info,
                                transport_info, info)
                self.packets.append(packet)

                # 安全地更新GUI
```

```
                self.root.after(0, self.add_packet_to_tree,
                                packet, protocol_name, ...)

            except Exception:
                if not self.sniffing:
                    break
                continue

    finally:
        if sock:
            sock.ioctl(socket.SIO_RCVALL,
                       socket.RCVALL_OFF)
            sock.close()
```

4.3 协议解析核心

IP头部分析：解析版本号（IPv4/IPv6）、头部长度、TTL、协议号、源IP地址、目的IP地址。根据协议号分发到不同的传输层解析器：

```
def parse_ip_header(self, data):
    if len(data) < 20:
        return None
    ip_header = struct.unpack('!BBHHHBBH4s4s', data[:20])
    version_ihl = ip_header[0]
    version = version_ihl >> 4          # 版本号（IPv4=4）
    ihl = (version_ihl & 0xF) * 4       # 头部长度
    ttl = ip_header[5]                  # 生存时间
    protocol = ip_header[6]             # 协议号（6=TCP,17=UDP,1=I
CMP）
    source_ip = socket.inet_ntoa(ip_header[8])   # 源IP
    dest_ip = socket.inet_ntoa(ip_header[9])      # 目的IP
    total_length = ip_header[2]         # 总长度
    return version, ihl, ttl, protocol, source_ip, dest_ip,
total_length, data[ihl:]
```

IP头部分析：解析版本号（IPv4/IPv6）、头部长度、TTL、协议号、源IP地址、目的IP地址。根据协议号分发到不同的传输层解析器。

TCP解析：解析源端口、目的端口、序列号、确认号、数据偏移和6个控制位标志，并通过端口号识别常见应用层协议（80-HTTP、443-HTTPS、53-DNS等）。

UDP解析：解析源端口、目的端口、数据报长度和校验和。

ICMP解析：解析类型和代码，识别Echo Request、Echo Reply、Destination Unreachable等常见ICMP消息类型。

ARP解析：解析硬件类型、协议类型、操作码（Request/Reply）以及发送方和目标方的MAC地址和IP地址。

```
def parse_arp_header(self, data):
```

```
    if len(data) < 28:
        return None
    arp_header = struct.unpack('!HHBBH', data[:8])
    operation = arp_header[4]
    sender_mac = ':'.join('{:02x}'.format(b)
                  for b in data[8:14]).upper()
    sender_ip = socket.inet_ntoa(data[14:18])
    target_mac = ':'.join('{:02x}'.format(b)
                  for b in data[18:24]).upper()
    target_ip = socket.inet_ntoa(data[24:28])
    return operation, sender_mac, sender_ip, target_mac,
target_ip
```

4.4 界面更新机制

由于Tkinter不是线程安全的，嗅探线程不能直接操作GUI组件。程序采用root.after(0, callback)机制，将GUI更新操作调度到主线程执行。每次捕获到一个数据包后，嗅探线程调用self.root.after(0, self.add_packet_to_tree, ...)将数据包添加到列表，确保线程安全。

```
# 创建并启动嗅探线程（在start_sniffing中）
self.sniffer_thread = threading.Thread(
    target=self.sniff_packets, daemon=True)
self.sniffer_thread.start()
```

4.5 筛选与搜索机制

协议筛选通过Combobox下拉框选择ALL/TCP/UDP/ICMP，搜索框支持关键词实时过滤。当筛选条件变化时，遍历所有已捕获的数据包，匹配协议类型和搜索关键词后重新填充列表。统计面板始终显示所有数据包的统计信息，不受筛选影响。

4.6 数据导出功能

程序支持将捕获的数据包导出为文本文件，导出内容包含数据包序号、时间戳、协议类型、长度和详细信息，便于后续离线分析。

```
def export_capture(self):
    if not self.packets:
        messagebox.showinfo("提示", "没有可导出的数据包")
        return

    from tkinter import filedialog
    filename = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Text files", "*.txt"),
                   ("All files", "*.*")])

    if filename:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(f"网络嗅探器捕获数据\n")
            f.write(f"导出时间: {datetime.now()}\n")
```

```
            f.write(f"数据包总数: {len(self.packets)}\n")
            f.write("="*100 + "\n\n")
            for packet in self.packets:
                f.write(f"No. {packet.number}\n")
                f.write(f"Time: {packet.relative_time}\n")
                f.write(f"Length: {len(packet.raw_data)}
bytes\n")
                f.write(f"Info: {packet.info}\n")
                f.write("-"*100 + "\n")
        messagebox.showinfo("成功",
            f"已导出 {len(self.packets)} 个数据包")
```

4.7 Ping工具实现

Ping工具在独立线程中调用subprocess.Popen执行系统ping命令。执行期间通过after()机制实时更新结果文本区域，完成后自动恢复按钮状态。默认发送4个ICMP包，超时时间为30秒。

```
def start_ping(self):
    host = self.ping_entry.get().strip()
    if not host:
        return
    self.ping_btn.config(state=tk.DISABLED)
    ping_thread = threading.Thread(
        target=self.do_ping, args=(host,), daemon=True)
    ping_thread.start()

def do_ping(self, host):
    try:
        cmd = ['ping', '-n', '4', host]
        process = subprocess.Popen(
            cmd, stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True, encoding='gbk',
            errors='replace')
        stdout, stderr = process.communicate(timeout=30)

        self.root.after(0, self.ping_text.insert,
                        tk.END, stdout if stdout else '')
        self.root.after(0, self.ping_btn.config,
                        {'state': tk.NORMAL})
    except subprocess.TimeoutExpired:
        self.root.after(0, self.ping_text.insert,
                        tk.END, 'Ping 请求超时\n')
        self.root.after(0, self.ping_btn.config,
                        {'state': tk.NORMAL})
```

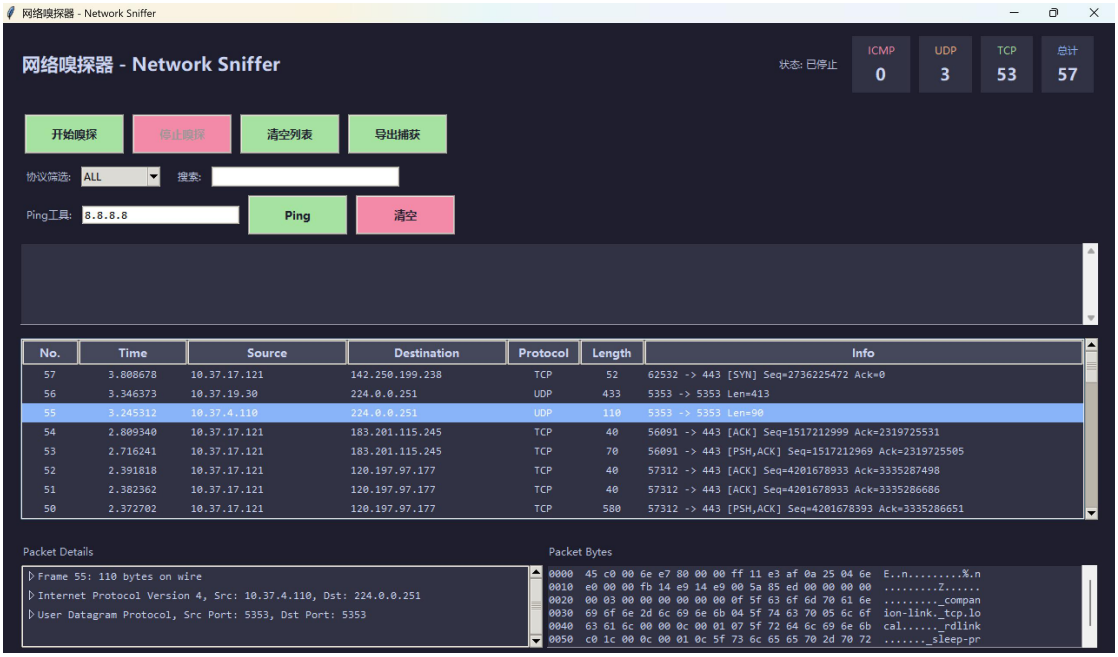
六、功能测试与分析

1、管理员权限测试

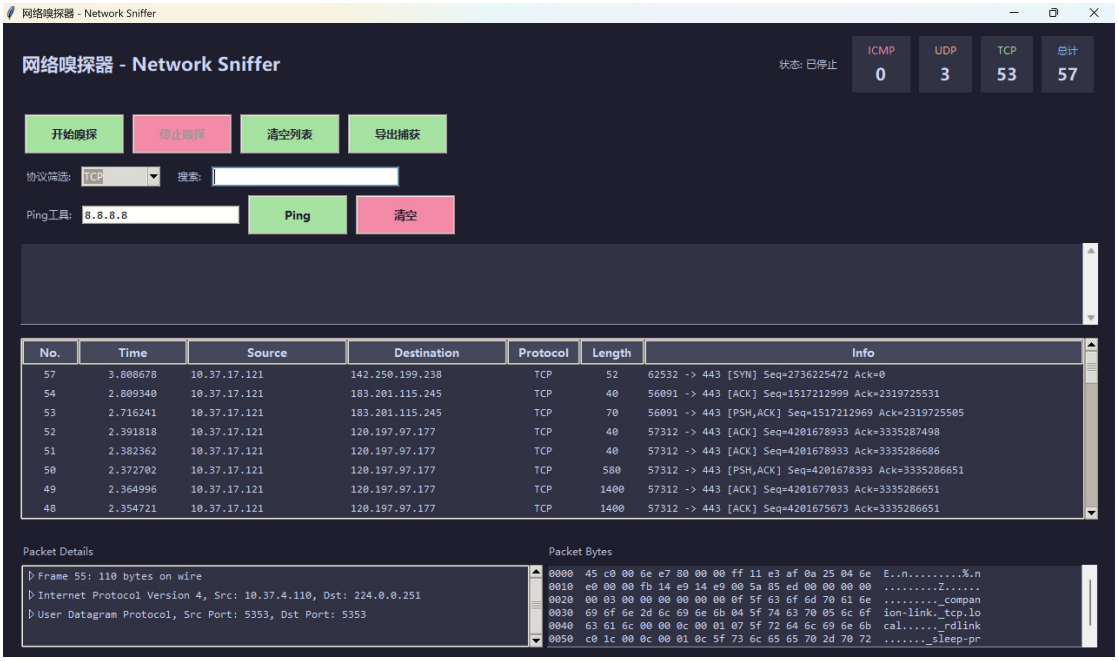
以普通用户身份运行程序时，弹出"请以管理员身份运行此程序"的错误对话框，程序正常退出。
以管理员身份运行程序时，程序正常启动，进入主界面。

2、数据包捕获功能测试

点击"开始嗅探"按钮后，状态栏显示"正在嗅探...", 数据包列表开始实时显示捕获到的网络数据包。测试结果如下：

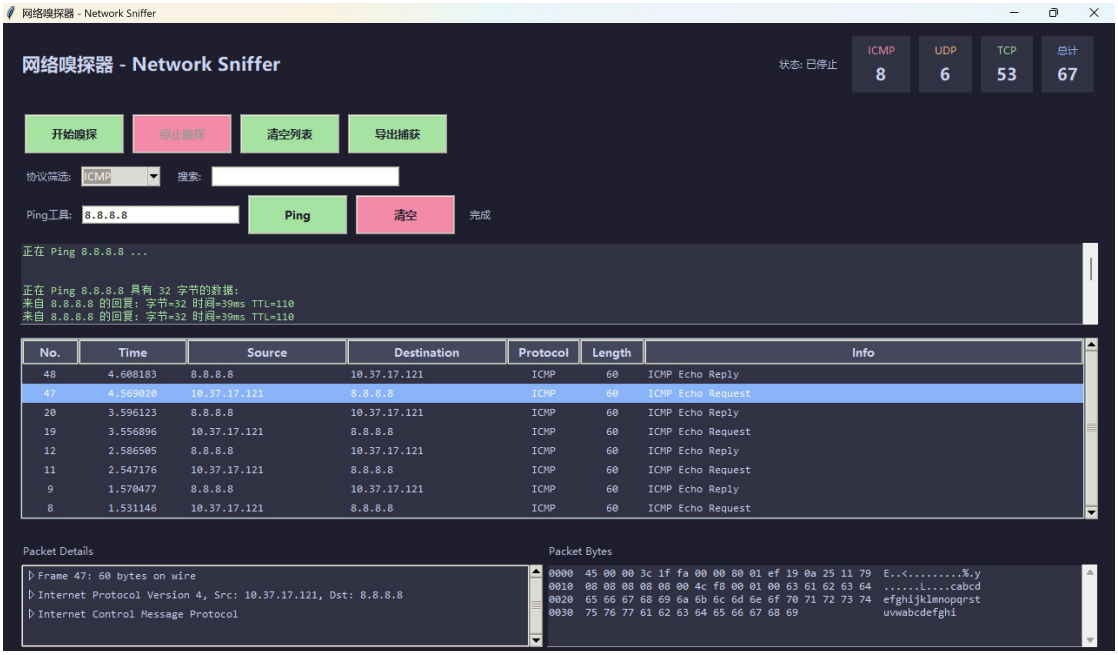


4、协议筛选与搜索测试



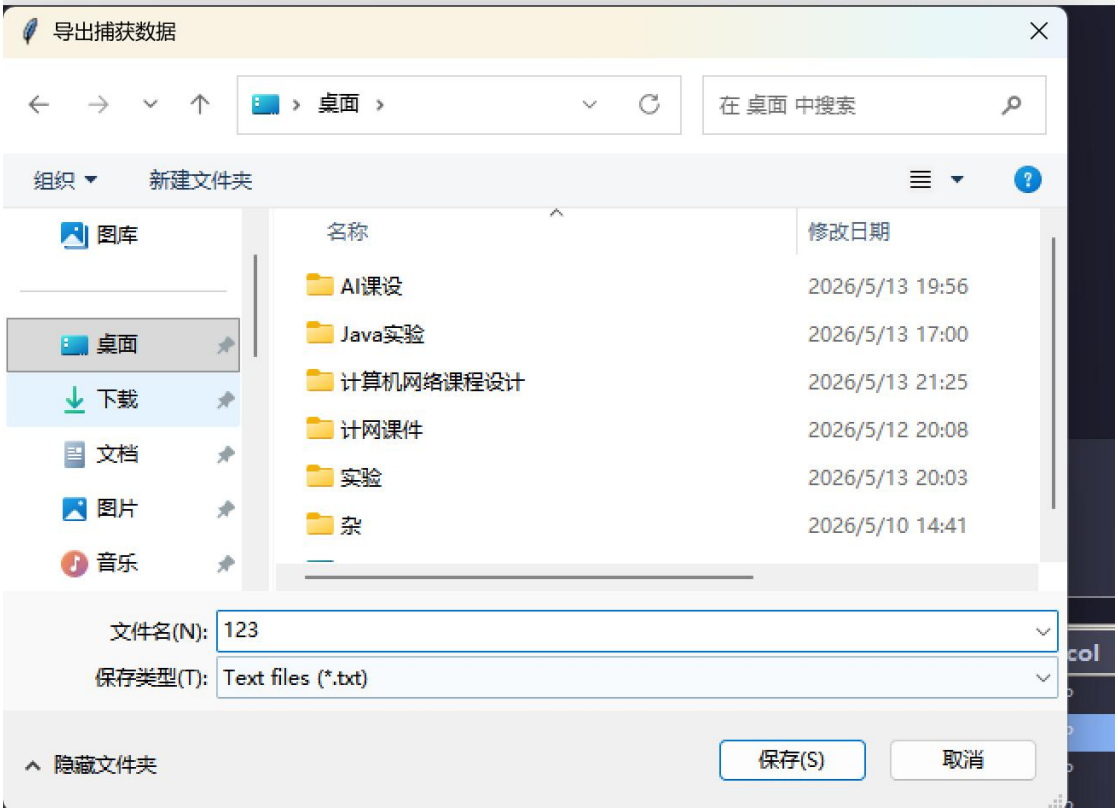
5、Ping工具测试

向8.8.8.8发送Ping请求，成功返回4个回复，Ping结果区域正确显示往返时间和TTL值。嗅探器同时捕获到ICMP Echo Request和Echo Reply数据包，验证了ICMP协议解析的正确性。



6、数据导出测试

点击"导出捕获"按钮，选择保存路径，成功导出包含所有数据包详细信息的文本文件。导出的文件内容格式正确，包含序号、时间、协议类型、长度和详细信息。



七、心得体会

通过本次课程设计，我深入理解了计算机网络协议栈的分层架构和网络数据包的传输过程。从理论到实践，将课堂上学到的TCP/IP协议知识应用到实际编程中，收获颇丰。

在实现过程中，我遇到了以下几个关键问题：

第一，管理员权限问题。原始套接字在Windows系统下需要管理员权限才能创建，否则程序无法正常工作。通过ctypes模块调用Windows API检测权限，并在启动脚本中自动请求管理员权限，解决了这个问题。

第二，多线程编程的线程安全问题。数据包捕获是阻塞操作，不能放在主线程中执行，否则会导致GUI界面卡死。使用threading模块创建独立嗅探线程，并通过Tkinter的after()方法将GUI更新操作调度到主线程执行，实现了流畅的用户体验。

第三，二进制数据包解析的复杂性。网络协议头部是二进制格式，通过Python的struct模块和位运算，成功实现了IP、TCP、UDP、ICMP和ARP协议的逐层解析。这个过程加深了我对协议头部各字段含义的理解。

第四，图形界面设计。参考Wireshark的三栏布局，使用Tkinter构建了包含数据包列表、协议详情树和十六进制转储区的完整界面，掌握了GUI编程的基本方法。

通过本次课程设计，我不仅巩固了计算机网络的知识，还提升了Python编程能力和解决实际问题的能力。特别是对原始套接字的工作原理、TCP/IP协议栈的分层解析有了更深刻的认识，为今后从事网络相关工作打下了坚实的基础。

八、参考文献

1. Stevens W R. TCP/IP Illustrated, Volume 1: The Protocols. Addison-Wesley, 1994.
2. RFC 791 - Internet Protocol Specification. IETF, 1981.
3. RFC 793 - Transmission Control Protocol Specification. IETF, 1981.
4. RFC 768 - User Datagram Protocol Specification. IETF, 1980.
5. RFC 792 - Internet Control Message Protocol Specification. IETF, 1981.
6. RFC 826 - Ethernet Address Resolution Protocol. IETF, 1982.
7. Python官方文档 - socket模块：https://docs.python.org/3/library/socket.html
8. Python官方文档 - struct模块：https://docs.python.org/3/library/struct.html